

MovieExplorer — Complete Beginner's Guide

A full walk-through of a **Movie Explorer** app built with **React (Vite)**, written for someone who has just created a Vite starter folder and wants to understand every file, every concept, and every line of logic.

Table of Contents

1. [What is this project?](#)
 2. [Tech stack at a glance](#)
 3. [How the app works \(the big picture\)](#)
 4. [Step 1 — Create the project from scratch](#)
 5. [Step 2 — The folder structure](#)
 6. [Step 3 — Read the files one by one](#)
 7. [Core React concepts you will meet](#)
 8. [The TVMaze API explained](#)
 9. [Feature deep-dives](#)
 10. [Run, build, and preview](#)
 11. [Deploy to the internet](#)
 12. [Git: save & push your work](#)
 13. [Troubleshooting checklist](#)
 14. [Glossary](#)
 15. [How to convert this guide to PDF](#)
-

1. What is this project?

MovieExplorer is a small web app where a visitor can:

- 🏠 See a **home page** with a hero banner, popular shows, and a how-it-works section.
- 🔍 **Search** for a movie / TV show by **title** — results update as you type.
- 📁 **Browse** the full catalog of thousands of TV shows.
- 📄 See each show on a **card** with poster, title, rating (★) and release year (📅).
- 🔖 Click **See Details** to open a **modal** with a big image, summary, genres, rating, and more.

All the data comes from a **free public API** called [TVMaze](#).

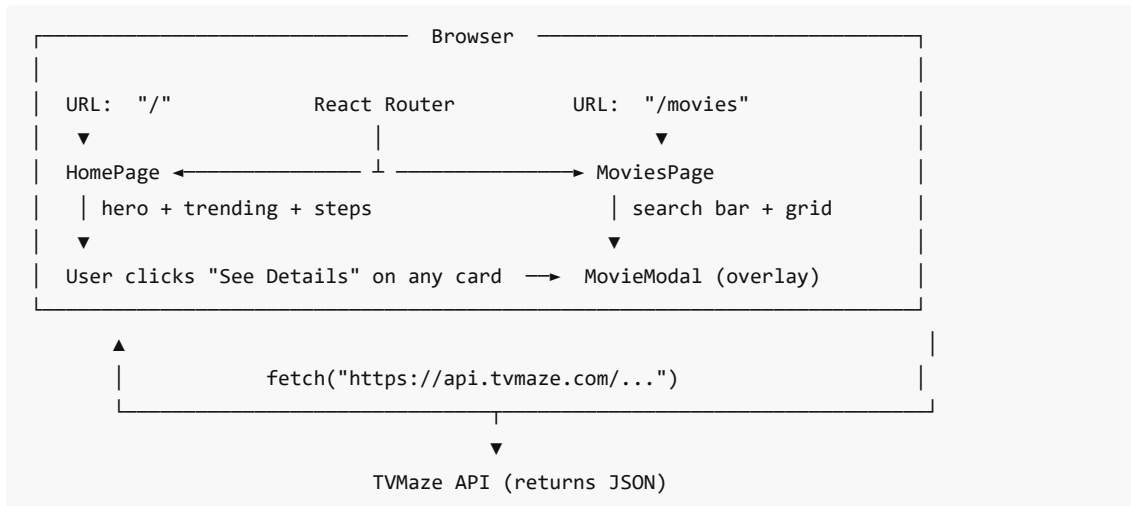
2. Tech stack at a glance

Layer	Technology	Why?
Language	JavaScript (ES modules)	The language of the web
UI library	React 19	Components + fast re-renders
Build tool	Vite	Instant dev server, fast builds
Router	react-router-dom	Multiple "pages" without reloading
Styling	Pure CSS (+ 2 Google Fonts)	Fully responsive, no framework needed
Data	TVMaze REST API	Free, no API key required

Linters	eslint	Catches bugs + bad style
---------	--------	--------------------------

★ **Fun fact:** Everything here is **static** — there is no backend. React asks TVMaze for data and draws the screen. That is why we can deploy it almost anywhere (Vercel, Netlify, GitHub Pages).

3. How the app works (the big picture)



Step by step:

1. The browser loads `index.html` → React starts in `main.jsx`.
2. `App.jsx` sets up the **router**: `/` shows `HomePage`, `/movies` shows `MoviesPage`.
3. Every page asks the **TVMaze API** for data.
4. While waiting, React shows **loading skeletons**.
5. When data arrives, React re-renders the movie card **grid**.
6. Clicking **See Details** opens a **modal** = a big overlay with full info.

4. Step 1 — Create the project from scratch

If you are starting with nothing, do this in your terminal:

```
# 1) Scaffold a new Vite + React project (this is the "starter folder")
npm create vite@latest movie-explorer -- --template react

# 2) Go inside the new folder
cd movie-explorer

# 3) Install React, Router and the dev tools
npm install
npm install react-router-dom

# 4) Start the dev server (open the printed URL, usually http://localhost:5173)
npm run dev
```

💡 What just happened?

- `npm create vite` downloaded a ready-made **starter folder** (React + Vite).
- `npm install` downloads all libraries named in `package.json` into the `node_modules` folder.
- `npm run dev` starts a live server that **auto-reloads** whenever you save a file.

Your starter folder will contain some template files (like `App.jsx` with a counter, `react.svg`, `vite.svg`, `App.css`, `index.css`). We then **replace** them with our own movie app files.

5. Step 2 — The folder structure

Here is the complete structure of the finished project:

```
movie-explorer/
|
├─ index.html           ← the ONLY html file; Vite uses it as an entry point
├─ package.json         ← project info + commands (scripts) + dependencies
├─ vite.config.js       ← Vite settings (React plugin)
├─ .gitignore           ← files Git should never upload (node_modules, dist...)
├─ README.md           ← project readme
|
├─ public/              ← files served "as-is" at the root of the site
|   ├── hero-bg.jpg     ← the background photo used in the hero banner
|   ├── favicon.svg     ← the little icon in the browser tab
|   └─ icons.svg        ← leftover template icons
|
└─ src/                 ← ALL the React code lives here
    ├── main.jsx         ← React entry point (starts the app)
    ├── App.jsx          ← the "router" – decides which page to show
    ├── index.css        ← global styles + colour/font variables (tokens)
    ├── App.css          ← styles for every component
    ├── assets/
    │   └─ no-poster.svg ← shown when a show has no poster image
```

```

|
├─ api/
│   └─ tvmaze.js      ← helper functions that call the TVMaze API
|
├─ hooks/
│   └─ useDebounceValue.js ← a custom hook that delays typing input
|
├─ components/        ← reusable building blocks (used in more than one place)
│   ├── Navbar.jsx
│   ├── Footer.jsx
│   ├── Poster.jsx
│   ├── MovieCard.jsx
│   ├── MovieModal.jsx
│   └─ SkeletonCard.jsx
|
└─ pages/              ← full screen-sized sections
    ├── HomePage.jsx   ← the landing page (URL: /)
    └─ MoviesPage.jsx  ← catalog + search (URL: /movies)

```

Every file and what it does

File	One-line job
index.html	Provides the empty <code><div id="root"></code> and loads the fonts
src/main.jsx	Mounts the <code><App /></code> component into <code>#root</code>
src/App.jsx	Router: chooses Home or Movies based on the URL
src/index.css	Colour palette, fonts and base styles for the whole site
src/App.css	Styling for the navbar, cards, modal, footer...
src/api/tvmaze.js	<code>searchShows(term, signal)</code> and <code>getShowCatalog(signal)</code>
src/hooks/useDebounceValue.js	Waits 450 ms after typing before searching
src/components/Navbar.jsx	Top bar: brand + links + "Browse catalog" button
src/components/Footer.jsx	Bottom column layout with links and copyright
src/components/Poster.jsx	Image that swaps to <code>no-poster.svg</code> if image is missing/broken
src/components/MovieCard.jsx	One card: poster, title, ★ rating, 📅 year, button
src/components/MovieModal.jsx	The big details pop-up window
src/components/SkeletonCard.jsx	Grey pulsing placeholder card while loading
src/pages/HomePage.jsx	Hero banner + trending scroller + steps
src/pages/MoviesPage.jsx	Search bar + result grid + empty/error states

6. Step 3 — Read the files one by one

6.1 index.html — the starting point

```
<!doctype html>
<html lang="en">
  <head>
    <link rel="preconnect" href="https://fonts.googleapis.com" />
    <link href="https://fonts.googleapis.com/css2?...&display=swap" rel="stylesheet" />
    <title>MovieExplorer — Discover Movies</title>
  </head>
  <body>
    <div id="root"></div>      <!-- React will fill this div -->
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

This is the **only HTML page** in the whole app. Vite loads `main.jsx`, which starts React, and React paints the entire UI inside the empty `<div id="root">`. The two Google Fonts (Inter, Sora) are linked here.

6.2 src/main.jsx — the entrance door

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

- `createRoot(...)` tells React "take control of the `#root` div".
- `.render(<App />)` paints the whole app.
- `<StrictMode>` is a React helper that double-runs effects during development to help you find bugs (it does nothing in the production build).

6.3 src/App.jsx — the router

```
import { BrowserRouter, Routes, Route } from 'react-router-dom'
import Navbar from './components/Navbar'
import Footer from './components/Footer'
import HomePage from './pages/HomePage'
import MoviesPage from './pages/MoviesPage'
import './App.css'

function App() {
  return (
```

```

    <BrowserRouter>
      <Navbar />
      <Routes>
        <Route path="/" element={<HomePage />} />
        <Route path="/movies" element={<MoviesPage />} />
      </Routes>
      <Footer />
    </BrowserRouter>
  )
}

export default App

```

- `<BrowserRouter>` watches the URL.
- `<Routes>` + `<Route path="...">` is like an **if / else for URLs**:

URL "/" → show HomePage
 URL "/movies" → show MoviesPage
 anything else → still HomePage (React Router default behaviour)

- **Navbar and Footer are always visible** — only the middle part changes.
- `export default App` lets `main.jsx` import it.

📌 **Beginner note:** `import Navbar from './components/Navbar'` looks for a file `components/Navbar.jsx` (or `.js / .jsx`) inside `src/` and gives us the component name `Navbar`.

6.4 `src/index.css` & `src/App.css`

`index.css` holds the **design tokens** — values used again and again:

```

:root {
  --bg: #0a0c11;      /* near-black background */
  --gold: #efb84b;     /* the golden accent colour */
  --ink: #f2f5fa;      /* main text colour */
  --muted: #9aa6c0;    /* secondary text */
  --radius: 16px;
}

```

These variables are then used everywhere in `App.css`:

```

.movie-card {
  border-radius: var(--radius); /* = 16px */
  background: var(--surface);
}

```

`App.css` contains the styling for every component — navbar, buttons, hero, cards, modal, footer, plus:

- `@media` rules that make the layout **responsive** (desktop grid → mobile single column).
- `@keyframes` for loading **shimmer** and modal **fade/slide** animations.
- a `prefers-reduced-motion` block that disables animations for users who ask for fewer animations.

6.5 src/api/tvmaze.js — talking to the API

```
const BASE_URL = 'https://api.tvmaze.com'

async function fetchJson(url, signal) {
  const response = await fetch(url, { signal })
  if (!response.ok) {
    throw new Error(`The request failed with status ${response.status}.`)
  }
  return response.json()
}

export function searchShows(term, signal) {
  const query = encodeURIComponent(term.trim())
  return fetchJson(`${BASE_URL}/search/shows?q=${query}`, signal).then((items) =>
    items.map((item) => item.show),
  )
}

export function getShowCatalog(signal) {
  return fetchJson(`${BASE_URL}/shows`, signal)
}
```

- `fetch(url)` is a built-in browser function that downloads data from a URL.
- `await` pauses the function until the download finishes (a **Promise**).
- `if (!response.ok)` → if the server returns an error status (404, 500...), we throw an error.
- `.json()` turns the raw text into a JavaScript object.
- **Search shape:** TVMaze `/search` returns `[{ show: {...} }, { show: {...} }]`, so we `.map()` each item to just `item.show`.
- The `signal` parameter lets React **cancel** the request when the user types something new (see 9.1).

6.6 src/hooks/useDebounceValue.js — the "wait" hook

```
import { useEffect, useState } from 'react'

export function useDebounceValue(value, delay = 400) {
  const [debounced, setDebounced] = useState(value)

  useEffect(() => {
    const timeout = setTimeout(() => {
      setDebounced(value)
    }, delay)
    return () => clearTimeout(timeout)
  }, [value, delay])

  return debounced
}
```

Why debounce? Searching on *every* keystroke (e.g. "b" → "br" → "bre") would fire hundreds of API calls. Instead we wait until the user **pauses 400 ms**, then search once. The `useEffect` `cleanup` (`clearTimeout`) cancels the

previous wait whenever the value changes.

6.7 src/components/Poster.jsx — safe images

```
import { useState } from 'react'
import noPoster from '../assets/no-poster.svg'

export default function Poster({ show, className = '', size = 'medium' }) {
  const url = show.image?.[size] || show.image?.medium || null
  const [broken, setBroken] = useState(false)

  if (!url || broken) {
    return <img className={` ${className} poster-fallback` } src={noPoster} alt={`Poster for ${show.name}`} />
  }

  return (
    <img
      className={className}
      src={url}
      alt={`Poster for ${show.name}`}
      loading="lazy"
      onError={() => setBroken(true)}
    />
  )
}
```

- Many shows have **no image** → we detect that and show a friendly fallback poster.
- `onError` handles the case where the URL exists but the download **fails** → swap to the fallback.
- `loading="lazy"` tells the browser to only download images near the screen (performance win).
- `show.image?.medium` is **optional chaining** — if `image` is `null`, don't crash, just give `undefined`.

6.8 src/components/MovieCard.jsx — one card

```
export default function MovieCard({ show, onSelect }) {
  return (
    <article className="card">
      <div className="card-media">
        <Poster show={show} size="medium" className="card-poster-img" />
        {genres?.[0] && <span className="card-genre">{genres[0]}</span>}
      </div>
      <div className="card-content">
        <h3 className="card-title">{name}</h3>
        <p className="card-meta">★ {score(show)} • 📅 {premierYear(show)}</p>
        <button className="btn btn-ghost card-action" onClick={() => onSelect(show)}>
          See Details
        </button>
      </div>
    </article>
  )
}
```

```
)  
}
```

- It receives **two props**: `show` (the data) and `onSelect` (a callback = a function to call when the button is clicked).
- Helpers `premierYear(show)` and `score(show)` turn the API data into a year and a rating string.
- The grid recipe lives in CSS: `grid-template-columns: repeat(auto-fill, minmax(200px, 1fr))` → automatically 1 to 5+ columns based on screen width.

6.9 `src/components/SkeletonCard.jsx` — the loading placeholder

```
export default function SkeletonCard() {  
  return (  
    <div className="card skeleton-card" aria-hidden="true">  
      <div className="skeleton skeleton-media" />  
      <div className="card-content">  
        <div className="skeleton skeleton-line skeleton-line-long" />  
        <div className="skeleton skeleton-line skeleton-line-short" />  
        <div className="skeleton skeleton-btn" />  
      </div>  
    </div>  
  )  
}
```

While the API is loading, we show a row of these grey shapes that gently **shimmer**. This is much nicer than an empty page or a simple "Loading..." text.

6.10 `src/pages/HomePage.jsx` — the landing page

Contains 3 pieces:

Piece	What it shows
Hero	Background photo + "DISCOVER MOVIES" + description + Explore Now button → <code>/movies</code>
Trending section	Fetches the catalog, sorts by rating, shows the top 10 in a horizontal scroll row
HowItWorks	Three small cards: Search → Compare → Explore

Interesting bits:

```
function Hero() {  
  return (  
    <section className="hero">  
      ...  
      <h1 className="hero-title">Discover<br /><span>Movies</span></h1>  
      <p className="hero-lead">Explore and discover your favorite movies from around the  
world.</p>  
      <Link to="/movies" className="btn btn-primary btn-lg">Explore Now</Link>  
      ...  
    </section>  
  )  
}
```

```
)  
}
```

- `<Link to="/movies">` from React Router navigates to the Movies page **without reloading** the page.
- The trending part uses `useEffect` → `getShowCatalog()` → sort by `rating.average` → `.slice(0, 10)` (first ten), with a `cancelled` flag to avoid touching state after the component unmounts.

6.11 `src/pages/MoviesPage.jsx` — the search page (the star of the show)

```
export default function MoviesPage() {  
  const [term, setTerm] = useState('') // what the user typed  
  const debouncedTerm = useDebounceValue(term, 450) // the "delayed" value  
  
  const [shows, setShows] = useState([]) // the results  
  const [state, setState] = useState('loading') // loading | ready | error  
  const [errorMessage, setErrorMessage] = useState('')  
  const [selected, setSelected] = useState(null) // the show in the modal  
  
  useEffect(() => {  
    const controller = new AbortController()  
  
    async function loadShows() {  
      try {  
        const list = debouncedTerm  
          ? await searchShows(debouncedTerm, controller.signal)  
          : await getShowCatalog(controller.signal)  
        setShows(list)  
        setState('ready')  
      } catch (error) {  
        if (error.name === 'AbortError') return // user typed something new  
        setErrorMessage(error.message || 'Something went wrong.')  
        setState('error')  
      }  
    }  
  
    loadShows()  
  
    return () => controller.abort()  
  }, [debouncedTerm])  
  ...  
}
```

The logic flow:

```
user types "nar" → debounce 450ms → value becomes "nar"  
→ searchShows("nar") fetches https://api.tvmaze.com/search/shows?q=nar  
→ state = 'ready', shows = [list of shows]  
→ grid of <MovieCard> re-renders
```

State machine (three possible states):

State	Screen shows
loading	The shimmer skeleton grid (12 SkeletonCards)
ready	The movie grid — or a friendly "no matches" message
error	🚨 message + a Try again button

6.12 src/components/MovieModal.jsx — the details pop-up

It builds an overlay with:

- a **banner image** (the largest available image)
- the **title**
- ⭐ Rating: 8.5 | 📅 Release: 2024 + status
- **genre chips** (buttons with the genres of the show)
- a clean **summary** (HTML tags are stripped with a small `plainText` helper)
- a **fact grid**: Language · Type · Network · Premiered
- a **Close** button

Interaction requirements (from the assignment):

Action	Result
Click the ✕ button	closes
Press Esc	closes
Click the dark backdrop behind the modal	closes
Click inside the modal	stays open

How that is implemented:

```
useEffect(() => {
  closeRef.current?.focus() // accessibility: focus the close button
  const onKeyDown = (event) => { if (event.key === 'Escape') onClose() }
  document.body.style.overflow = 'hidden' // stop background scrolling
  document.addEventListener('keydown', onKeyDown)
  return () => { // cleanup when modal closes
    document.body.style.overflow = ''
    document.removeEventListener('keydown', onKeyDown)
  }
}, [onClose])
```

```
<div className="modal-layer" onClick={onClose}> // backdrop click closes
  <div className="modal" onClick={(e) => e.stopPropagation()}> // inner click does NOT
close
  ...content...
</div>
</div>
```

• `stopPropagation()` stops the click from "bubbling up" to the backdrop, so clicking inside the modal never closes it.

7. Core React concepts you will meet

7.1 Components & JSX

A **component** is a function that returns "HTML written inside JavaScript":

```
function Greeting({ name }) {  
  return <h1>Hello, {name}!</h1>  
}
```

- Starts with a **capital letter** → React treats it as a component.
- The `{ name }` syntax unpacks **props** (inputs).
- `{name}` inside JSX injects the JavaScript value.

7.2 Props (inputs)

```
<MovieCard show={show} onSelect={setSelected} />
```

`show` = data, `onSelect` = a callback (function). The card uses them but never changes them.

7.3 State with `useState` (memory)

```
const [term, setTerm] = useState('')
```

- `term` is the current value, `setTerm` is the only way to change it.
- Calling `setTerm(...)` tells React: "*re-render with the new value.*" That is how the grid updates when the search changes.

7.4 Effects with `useEffect` (side effects)

```
useEffect(() => {  
  // runs after the component is painted on screen  
  return () => { /* optional cleanup, runs before the next run / unmount */ }  
}, [dependencies])
```

Used for: fetching data, listening to keyboard/scroll events, locking body scroll when the modal is open. The **dependency array** decides *when* it runs.

<code>useEffect(..., [])</code>	→ runs once on mount
<code>useEffect(..., [term])</code>	→ runs again every time "term" changes
<code>useEffect(...)</code>	→ runs after EVERY render (rarely what you want)

7.5 Custom hooks

A regular function that uses React hooks (like `useState` , `useEffect`) and starts with `use` :

```
const debouncedTerm = useDebounceValue(term, 450)
```

This is just a way to **reuse logic** between components.

7.6 Routing (react-router-dom)

Component	Purpose
<code><BrowserRouter></code>	Wraps the app and watches the URL
<code><Routes></code>	Container for routes
<code><Route path="/movies" element={<MoviesPage />} /></code>	"When URL is /movies, show <MoviesPage />"
<code><Link to="/movies"></code>	Link that navigates without a full page reload
<code><NavLink to="/"></code>	A Link that also tells you if it's "active"

7.7 Fetching data (async/await & try/catch)

The cleanest pattern is **async/await inside a try/catch** — the style used throughout this project:

```
async function loadData() {
  try {
    const response = await fetch(url)    // pause until the download finishes
    const data = await response.json()   // parse the JSON
    setShows(data)                      // update the screen
  } catch (error) {
    setError(error.message)             // only runs if something above threw
  }
}
```

- `await` pauses the function until a **Promise** resolves (or rejects).
- If anything inside `try` **throws**, execution jumps straight to `catch`.
- The API helper (`src/api/tvmaze.js`) throws on bad HTTP statuses:

```
if (!response.ok) {
  throw new Error(`The request failed with status ${response.status}.`)
}
```

That thrown error is exactly what the `catch` blocks in `HomePage.jsx` and `MoviesPage.jsx` catch.

💡 The older style looked like `fetch(url).then(...).catch(...)` — the project started with that and was refactored to `try/catch` for readability. Both do the same thing.

`AbortController` + `signal` lets us cancel a fetch that is no longer needed (old search term, component closed).

7.8 Conditional rendering

```
{state === 'loading' && <SkeletonCard />}
{state === 'error' && <p>⚠️ {errorMessage}</p>}
{selected && <MovieModal show={selected} onClose={...} />}
```

`&&` means: "if the left side is truthy, render the right side."

8. The TVMaze API explained

Endpoints used

Endpoint	Returns	Used by
GET https://api.tvmaze.com/shows	A big array of all shows	Movies page default view + Home trending
GET https://api.tvmaze.com/search/shows?q=:query	Array of { score, show }	Movies page search

Example: searching girls

GET https://api.tvmaze.com/search/shows?q=girls

Response (simplified):

```
[
  {
    "score": 9.4,
    "show": {
      "id": 139,
      "name": "Girls",
      "premiered": "2012-04-15",
      "rating": { "average": 6.5 },
      "image": { "medium": "https://.../139.jpg", "original": "https://.../139-large.jpg" },
      "genres": ["Drama", "Comedy"],
      "summary": "<p>Hannah...</p>",
      "language": "English",
      "status": "Ended",
      "type": "Scripted"
    }
  }
]
```

Field	What we use it for
show.name	Title on the card and modal
show.image.medium / .original	Card poster / modal banner
show.rating.average	The ★ rating
show.premiered	Release date → year
show.summary	Modal overview (we strip the HTML)
show.genres	Genre chips + first genre tag on card
show.language, show.type, show.status, show.network	The fact grid

⚠ Some shows have **no image, no rating, or no summary** — the app checks every field with fallbacks (`?` , `-` , `N/A` , `no-poster.svg`).

9. Feature deep-dives

9.1 How search works (with cancellation)

```
type "g"
  → term = "g"                (input instantly)
  → debounce timer set 450ms
type "gi"
  → term = "gi"                (previous timer cancelled!)
  → debounce timer set 450ms
...user pauses 450ms...
  → debouncedTerm = "gi"
  → useEffect fires → searchShows("gi", signal)
  → old in-flight request (for "g") is aborted via controller.abort()
  → skeleton shown during the wait
  → results replace the grid
```

9.2 How the trending scroller works

1. On mount, HomePage fetches `GET /shows` (the whole catalog).
2. It sorts the array by `rating.average` (highest first).
3. `.slice(0, 10)` keeps the top ten.
4. They are rendered in `.scroller` — a `grid-auto-flow: column` container you can swipe/scroll horizontally (scrollbar hidden).

9.3 How the modal reuses components

- The **same** `<MovieCard>` is used on the Home trending row *and* the Movies grid.
 - The **same** `<MovieModal>` is used by both pages.
 - `selected` state holds *which* show is open; setting it to `null` closes the modal. That is why the modal is rendered with `{selected && <MovieModal .../>}` .
-

10. Run, build, and preview

Make sure you are inside the project folder:

```
cd movie-explorer

npm install      # only the first time – installs dependencies
npm run dev      # development server (hot reload) → http://localhost:5173
npm run build    # create the final production files in /dist
npm run preview  # serve the production build locally to test it
npm run lint     # check the code with oxlint
```

Command	What it does	When to use
<code>npm run dev</code>	Live dev server, instant reload	While coding

npm run build	Optimized static site in <code>dist/</code>	Before deploying
npm run preview	Serves <code>dist/</code> on your machine	Testing the build
npm run lint	Reports bugs/bad style	Before committing

11. Deploy to the internet

The project has **no server** — it is a folder of static files. Any static host works.

Option A — Vercel (fastest)

1. Go to <https://vercel.com> → Sign up with GitHub
2. Click “New Project” → import your repository
3. Vercel auto-detects Vite: Build command = `npm run build`
Output dir = `dist`
4. Press Deploy → you get a live URL like <https://movie-explorer.vercel.app>

Option B — Netlify

1. <https://netlify.com> → “Add new site” → “Import an existing project”
2. Pick your GitHub repo
3. Build command: `npm run build`
4. Publish directory: `dist`
5. Deploy → live URL

Option C — GitHub Pages

1. Add a file `vite.config.js` with:

```
export default defineConfig({
  plugins: [react()],
  base: '/Assignment_2/',
})
```
2. Push to GitHub
3. Repo → Settings → Pages → Source: GitHub Actions
(or push the `dist/` folder to the `gh-pages` branch)
4. Your site: https://<username>.github.io/Assignment_2/

💡 Why does `base` matter for GitHub Pages? Because that host serves the site from a **sub-folder** (`/Assignment_2/`), not the root. Vercel/Netlify serve from the root, so no change is needed there.

12. Git: save & push your work

```
cd movie-explorer
git init                # 1) turn this folder into a Git repository
git add .               # 2) stage all files (node_modules/dist are ignored)
git status              # 3) check what will be committed
git commit -m "first commit" # 4) save a snapshot
git branch -M main      # 5) name the current branch "main"
git remote add origin https://github.com/YOUR_USERNAME/Assignment_2.git # 6) connect to
GitHub
git push -u origin main  # 7) upload everything to GitHub
```

After that, when you change files:

```
git add .
git commit -m "describe what you changed"
git push
```

🗨 `node_modules` (all installed libraries) and `dist` must **never** be uploaded — the `.gitignore` file already tells Git to skip them. Anyone who clones your repo just runs `npm install` to get them back.

13. Troubleshooting checklist

Symptom	Likely cause	Fix
npm run dev fails	Never ran <code>npm install</code>	Run <code>npm install</code> first
Blank white screen	JS error in console	Open DevTools (F12) → Console → read the error
Images missing	No internet / show has no image	Poster fallback (<code>no-poster.svg</code>) kicks in
Search feels laggy	That is the debounce	Normal — it waits 450 ms on purpose
API error message	TVMaze down or blocked	Click Try again
Router: page not found on refresh	Needs host rewrite (SPA)	Vercel/Netlify handle it automatically
Fonts not loading	Offline	CSS falls back to system fonts
Build fails	Syntax error in a file	Run <code>npm run lint</code> and read the first error

14. Glossary

Term	Meaning
Component	A reusable React function that returns JSX
Props	Inputs passed into a component
State	Data that React remembers and re-renders when changed
Hook	React function starting with <code>use</code> (e.g. <code>useState</code> , <code>useEffect</code>)
Effect	Code that runs after a render (fetching, events, timers)
Debounce	Delay an action until the user stops typing for a moment
Promise	An object that will (eventually) have a value or an error
async / await	Cleaner syntax for working with Promises
API	A URL that returns data (here: JSON)
JSON	The data format APIs return

Endpoint	A specific API address, e.g. <code>/search/shows?q=girls</code>
AbortController	Lets you cancel an in-flight fetch
Router	Code that maps URLs to components
Modal	A window that pops over the page
Responsive	Layout adapts to screen width (CSS media queries)
Skeleton	Grey placeholder shown during loading
JSX	JavaScript + HTML-like markup written together
npm	The tool that installs/manages libraries

15. How to convert this guide to PDF

This file is plain Markdown, so almost any converter works:

Option 1 — VS Code (easiest)

1. Install the extension "Markdown PDF" (by yzane)
2. Right-click this file → "Markdown PDF: Export (pdf)"
3. Done — a PDF appears next to the file

Option 2 — Command line (md-to-pdf)

```
npx md-to-pdf GUIDE.md      # also installs Chromium automatically
```

Option 3 — Pandoc (LaTeX based)

```
pandoc GUIDE.md -o GUIDE.pdf --pdf-engine=wkhtmltopdf
```

Tips for clean output

- The `<div style="page-break-before: always;"></div>` lines are **page-break hints** — most converters (md-to-pdf, Markdown PDF) respect them and start a new page there.
- Tables render best in wide layouts; avoid shrinking the PDF preview too much.
- If your converter shows code blocks overflowing, set a smaller base font (e.g. md-to-pdf `--md-to-pdf-option baseFontSize=11`).

✓ End of guide

You now know:

- how to scaffold a Vite + React project,
- how every file in `movie-explorer` works,
- how React state, effects, props and routing work in practice,
- how the TVMaze API integrates search and the full catalog,

- how the cards and the details modal are built,
- how to run, build, deploy, and share the project with Git.

Next step: open `src/components/MovieModal.jsx` , change something small (e.g. the text of the Close button), save, and watch the dev server live-reload it. Trial and error is the best teacher.